

Programmazione Concorrente e multithread Parte 3

Emanuele Ing. Benatti

Problemi classici

- **Algoritmo del banchiere**
- **Problema dei filosofi**
- **Algoritmo produttore/consumatore**
- **Algoritmo lettori/scrittori**

Algoritmo del banchiere

- L'algoritmo è stato proposto da Dijkstra (1965) imita il comportamento di un banchiere nei confronti delle richieste dei clienti
- (i clienti sono le risorse)
- Tratta risorse multiple
- Ogni processo deve dichiarare il numero massimo di risorse necessarie
- Se un processo richiede una risorsa, se non è disponibile rimane in attesa
- Dopo che un processo ha ottenuto tutte le risorse necessarie, le deve rilasciare in un tempo finito

Esempi

Si suppongono 10 risorse totali

Nome	U	M	N
Anna	1	6	5
Barbara	1	5	4
Carlo	2	4	2
Davide	4	7	3
Disp.	2		

Quale dei due stati è sicuro?

Nome	U	M	N
Anna	1	6	5
Barbara	2	5	3
Carlo	2	4	2
Davide	4	7	3
Disp.	1		

Problema dei filosofi

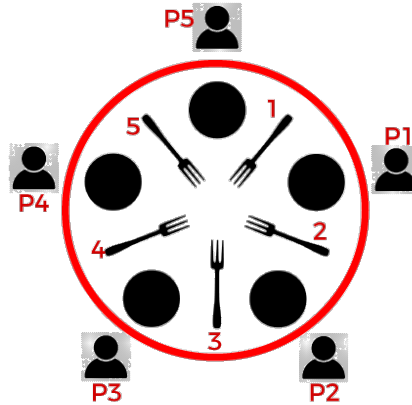
Si immagini che la vita di un filosofo consista di periodi alterni di mangiare e pensare, e che ciascun filosofo abbia bisogno di due forchette per mangiare, ma che le forchette vengano prese una per volta. Dopo essere riuscito a prendere due forchette il filosofo mangia per un po', poi lascia le forchette e ricomincia a pensare.

E' il classico problema che spiega un possibile stallo (deadlock) perchè se tutti i filosofi vogliono mangiare insieme non tutti riusciranno a prendere le forchette e saranno tutti in attesa infinita...



Problema dei filosofi

Problema: trovare una strategia di gestione delle risorse che consenta ad ogni filosofo di continuare all'infinito l'esecuzione delle due attività.



Problema dei filosofi (soluzione)

Associo ad ogni bacchetta un semaforo (binario)

Semaphore b[5]; // array di semafori

Semafori sono inizializzati a 1

b[i] = new Semaphore(1);

....

b[4] = new Semaphore(1);

Il filosofo i, prima di iniziare a mangiare, acquisisce in sequenza:

la bacchetta i (bacchetta sx)

la bacchetta (i+1)%5 (bacchetta dx)

Un filosofo non può prendere prima la bacchetta di destra invece di quella di sinistra!

Per quale motivo
effettivamente
questo funziona?

Algoritmo produttore/consumatore

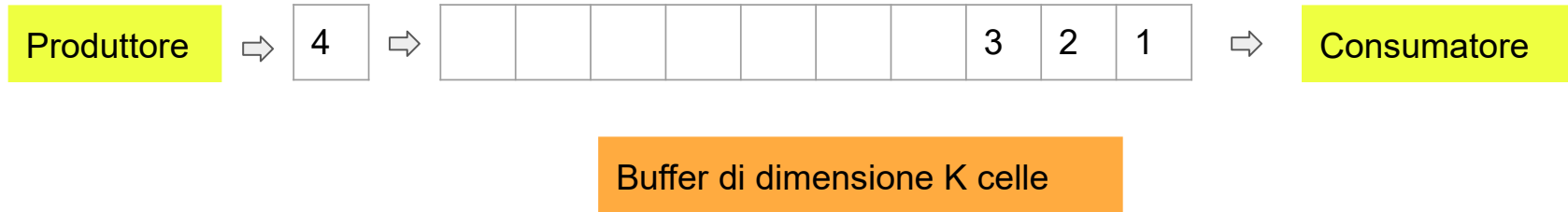
Il **problema del produttore-consumatore** detto anche problema del buffer limitato è un problema dove ci sono due processi che condividono un buffer comune di dimensione fissata dove:

- Un processo produttore inserisce dati all'interno,
- Un processo consumatore legge i dati prodotti, rimuovendoli di volta in volta dal buffer.

Problema:

- Come assicurare che il produttore si fermi di produrre quando il buffer è pieno e riprenda l'esecuzione quando almeno un blocco di memoria è libero?
- Come assicurare che il consumatore non cerchi dati quando il buffer è vuoto e venga avvisato quando qualche dato è disponibile?

Problema Produttore/Consumatore



Implementazione dell'algoritmo tramite un array circolare

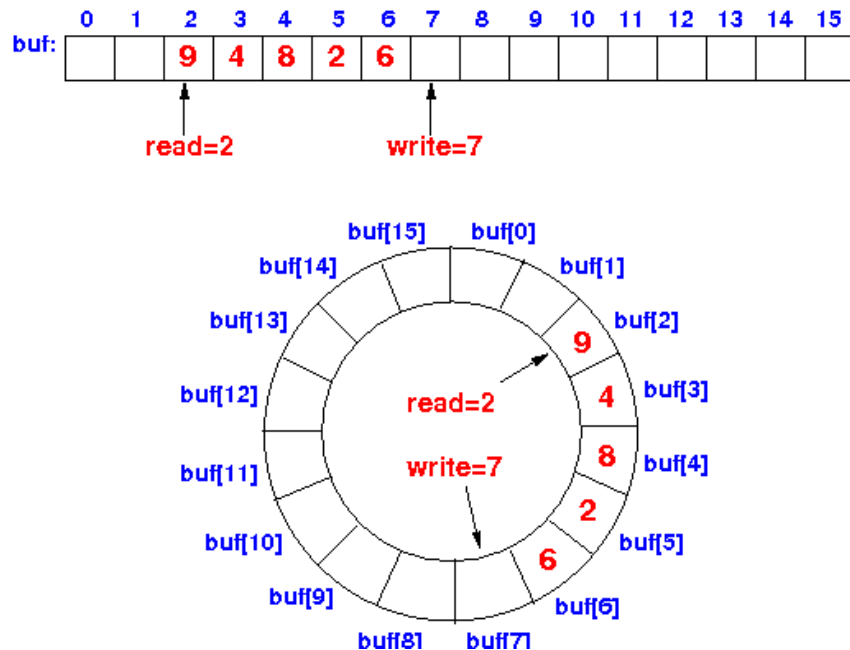
Un array circolare è una struttura dati dove ogni elemento dell'array c'è una continuità logica.

In pratica dopo aver scritto nell'ultimo elemento si riparte dal primo.

Quindi un produttore può continuare a scrivere come se l'array fosse infinito, utilizzando l'operatore di modulo per riportarsi in una posizione effettiva dell'array

es. $\text{buf}[\text{posiz} \% \text{dim}] = 7$ // scrivo 7 in una posizione precisa dell'array.

Se posiz è 64 e dim è 16 come nell'esempio scriverò in quale posizione 7?



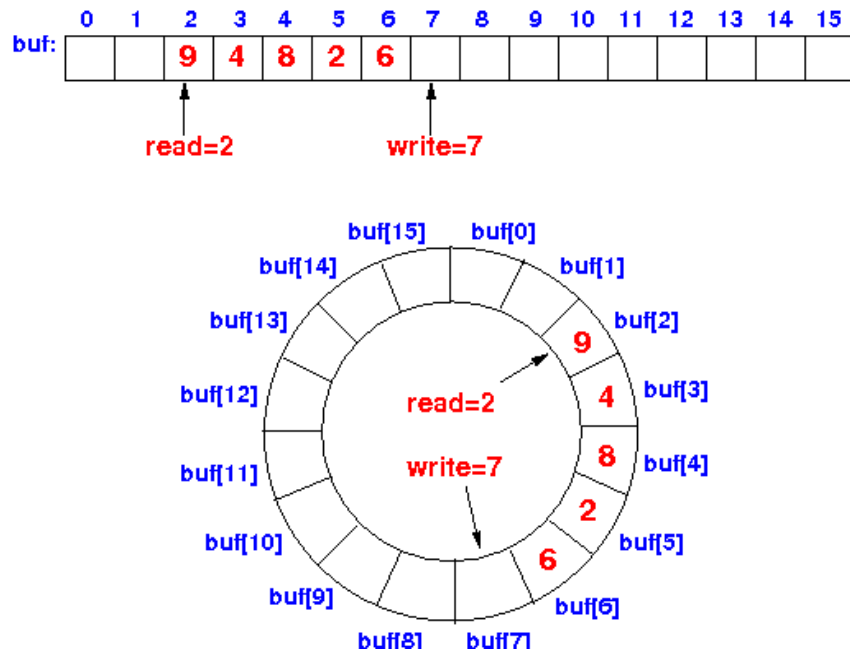
Implementazione dell'algoritmo tramite un array circolare

- Come assicurare che il produttore si fermi di produrre quando il buffer è pieno?

Attraverso un semaforo che dichiari se l'array circolare è pieno. Quando il consumatore lo avrà liberato di almeno una cella di memoria il produttore potrà scrivere

- Come assicurare che il consumatore non cerchi dati quando il buffer è vuoto?

Attraverso un semaforo che dichiari se l'array circolare è vuoto. Quando il produttore lo avrà riempito di almeno una cella di memoria il consumatore potrà leggere.



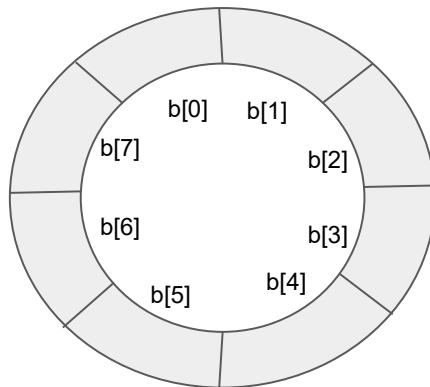
Operazioni su un array circolare

Dato il seguente array



b[0] b[1] b[2] b[3] b[4] b[5] b[6] b[7]

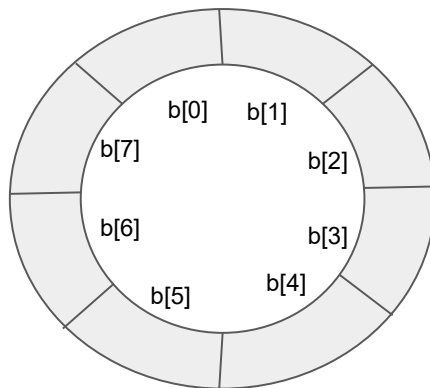
Può essere visto così...



Operazioni su un array circolare

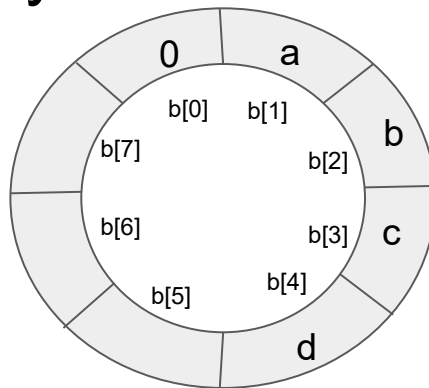
La posizione successiva di
 $b[i]$ è $b[i+1 \% \text{DIM}]$

Se ad esempio $i=7$
il successivo è $b[i+1 \% \text{DIM}]$
 $= b[0]$

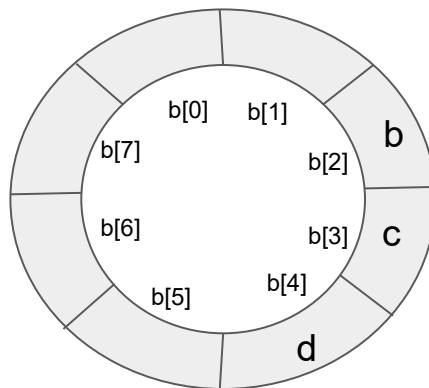


Operazioni su un array circolare

Inserisco 5 elementi

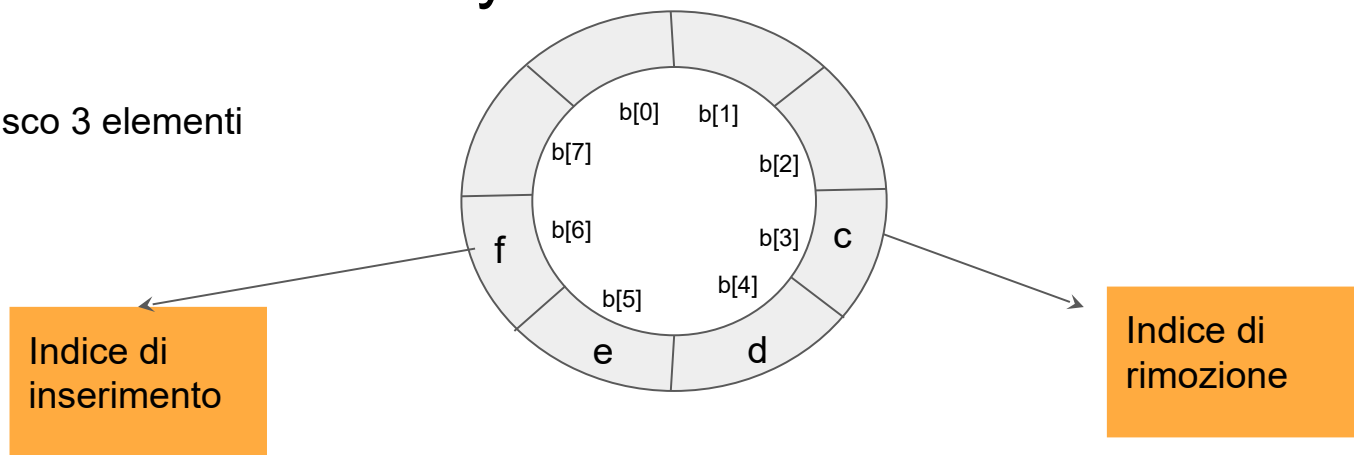


Rimozione di due
elementi

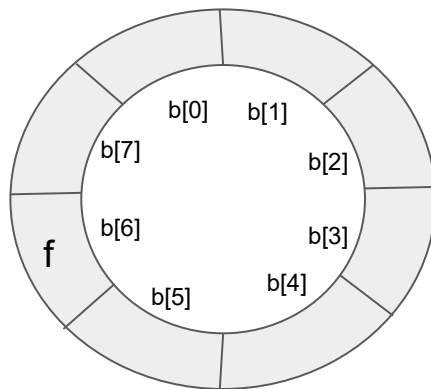


Operazioni su un array circolare

Inserisco 3 elementi



Rimozione di tre elementi



La classe BufferCircolare - tipo Integer

```
public class BufferCircolare{  
    public int N;  
    public int B[];  
    public int in=0; // elemento da inserire  
    public int out=0; // elemento da estrarre  
  
}
```

metodi

```
void inserisci(int Elem) {  
    b[in] = elem;  
    in=(in+1)%N;  
}  
int rimuovi() {  
    tmp=B[out];  
    out=(out+1)%N;  
    return tmp;  
}
```


Schema di soluzione con i semafori

Partendo dalla classe Buffer
Circolare uso due semafori

N = Dimensione del buffer
Due semafori su due variabili
P = Numero posizioni piene
V = Numero posizioni vuote
Dove $P=0$ e $V = N$ all'inizio!

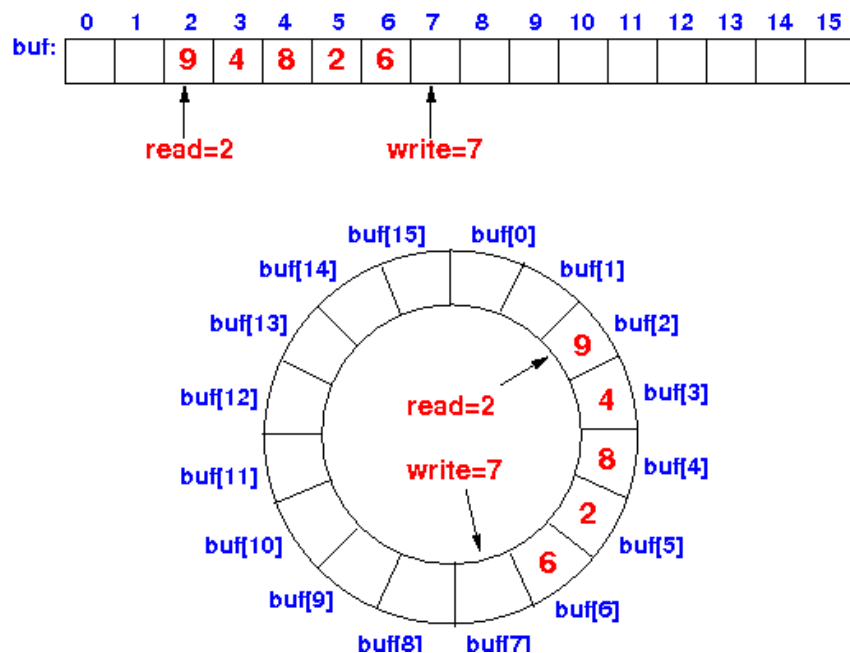
```
Produttore (inserisco un valore K)  
Down(V)  
Inserisco K  
UP(P)
```

```
Consumatore (estraggo un valore J)  
Down(P)  
Estraggo J  
UP(v)  
Uso J
```

Se non ci sono posizioni vuote o non ci sono elementi da estrarre questo schema è bloccante....

Bozza di soluzione

```
public void inserisci(int elem){  
    posVuote.acquire();  
    buff [index]=elem;  
    index=(index+1) %N;  
    posPiene.release();  
}  
  
public int estrai(){  
    posPiene.acquire();  
    int tmp = buff [out];  
    out=(out+1) %N;  
    posVuote.release();  
}
```



Algoritmo lettori scrittori

Problema:

- Più thread condividono l'accesso ad una memoria condivisa.

Alcuni thread sono interessati solo a leggere l'area di memoria e non c'è alcuna necessità di consumare una volta sola i dati (i dati possono essere letti più volte)

- Altri thread devono scrivere (modificare) l'area

Algoritmo lettori scrittori

Esempio: un database molto grande con tanti processi in competizione per leggere o scrivere

Molti possono leggere contemporaneamente, ma se un processo sta scrivendo (modifica) nessuno può leggere!

(Evitare la violazione delle proprietà ACID, vedi Informatica)

Algoritmo lettori scrittori

Il problema consiste nel sincronizzare l'accesso alla risorsa condivisa in modo che valgano i vincoli:

- 1) Nessun thread deve poter accedere ai dati, nè per scrivere nè per leggere, se è in corso un'operazione di scrittura, (ci sarebbe una corruzione dei dati)
- 2) più thread lettori **potrebbero** poter leggere contemporaneamente se non è in corso una scrittura

Soluzione

- Se non è necessario che piu' thread leggono dati contemporaneamente è sufficiente un unico semaforo dove un unico thread in lettura o scrittura può accedere ai dati condivisi.
- Se è necessario che piu' thread possano leggere in contemporanea servono due semafori.
 - il primo per gestire la scrittura esclusiva di un unico thread,
 - il secondo per garantire la lettura ad n Thread

Soluzione (cenni)

```
writer() {  
    lockscrittura.down();      //Blocco l'area di memoria per la scrittura, se ci sono lettori è bloccante  
    // scrittura della memoria  
    lockscrittura.up()        //La rilascio in modo che i lettori possano accedere  
}
```

```
reader() {  
    locklettura.down()  //sezione critica sui lettori  
    readcount++;        //Aumenta il numero di lettori attivi  
    if (readcount == 1) //Se è il primo lettore attivo  
        lockscrittura.down(); // blocco gli scrittori (o attendo che termini di scrivere lo scrittore)  
    locklettura.up();    //  
  
    //lettura  
  
    locklettura.down()  //sezione critica sui lettori  
    readcount--;        //un lettore se ne va  
    if (readcount == 0) //se è l'ultimo lettore  
        lockscrittura.up(); //abilito i lettori a leggere  
    locklettura.up();    //  
}
```

Esercizio Algoritmo Produttore e Consumatore

Scrivi un programma Java che implementa la seguente simulazione:

Un processo produttore produce in un tempo casuale che va da 100 a 800 ms un valore compreso tra 10 e 100 corrispondente alla velocità di una autovettura in corsa (nella generazione random, fatta nella funzione `generaVelocitaRandom()` per ipotesi non possono essere più o in meno di 10 km orari tra un numero generato casualmente e il precedente).

Un processo consumatore ogni 450 ms fissi va a leggere se esiste un valore.

Il processo consumatore aggiorna una variabile media corrispondente alla velocità dell'autovettura e la stampa immediatamente.

Nota bene: il pgm è automatico in quanto **DEVE** restituire una stampa dei vari in tempo reale.